B

BANTU
Programming Language

v1.2.2 — Stable Release

Official Language Guide & Toolchain Reference

include modules · PostgreSQL · MySQL · WebRTC · VSCode · cross-platform installer

Assey Silivestir Peter

github.com/AsseySilivestir/Bantu

Table of Contents

1. Introduction to Bantu

Bantu is a high-level, dynamically-typed programming language implemented as a tree-walking interpreter in C++17. The entire toolchain — interpreter, package manager, HTTP server, WebRTC engine, SQLite/PostgreSQL/MySQL drivers, project scaffolding, and cross-platform desktop installer generator — ships as a single ~660 KB static binary with zero runtime dependencies. Drop it on PATH and **bantu init** just works, on any 64-bit Linux or Windows machine.

Bantu was designed for offline-first developer environments: schools, hackathons, embedded systems, low-bandwidth regions, and any situation where installing Node.js or Python is impractical. The language borrows syntactic familiarity from JavaScript (curly braces), PHP ($-prefixed variables), and Python (dot access) while keeping the semantic surface small enough to fit in one human-readable manual.

This v1.2.2 stable release is a major step forward. New features include a real module system (`include` keyword with path resolution and cycle detection), optional real PostgreSQL / MySQL / WebRTC drivers (compile-time flags), a VSCode extension with syntax highlighting, autocomplete, hover hints, and a blue-B file icon, and a new **bantu installer** command that produces cross-platform desktop installers (**.deb** for Linux, **.exe** for Windows, **.app** for macOS) from any Bantu project — so you can ship a Bantu app to a friend's laptop and it just runs, no install step needed.

1.1 Design Principles

Five principles guide Bantu's evolution. They explain what the language does well and where it deliberately trades raw compute speed for simplicity and portability.

Principle	What it means in practice
Single-binary distribution	One executable. No runtime, no package manager install, no system Python. Copy bantu
Offline-first	The local package registry is a folder. bantu init, bantu add, bantu install
Batteries included	HTTP server, HTTP client, SQLite, PostgreSQL, MySQL, JSON, WebRTC, STUN/TURN — all b
Read-the-manual simple	The full language fits in this 40-page guide. A developer should be productive in Bantu after a si
Ship-anywhere deployment	The new bantu installer command packages your app into a single .deb / .exe / .app tha

1.2 What's New in v1.2.2

This release ships four major additions, several quality-of-life improvements, and a refreshed toolchain surface. The release tag is `v1.2.2` on GitHub `github.com/AsseySilivestir/Bantu`.

1.3 What's New in v1.2.2 vs v1.2.1

v1.2.2 is a **drop-in maintenance release** on top of v1.2.1. There are no language changes and no breaking API changes — every v1.2.1 program runs unchanged. The release focuses on robustness, operational ergonomics for the `include` module system, and adds a new **cross-platform installer generator**. Full changelog is in `CHANGELOG.md`.

Change	Description
bantu installer command	[NEW] command generates a cross-platform desktop installer for any Bantu project. Linux → .deb

Change	Description
Path canonicalization	Include paths are now resolved through realpath() (POSIX) / <font face="C
Cycle-guard diagnostic	Circular includes used to be silently skipped. They now print Skipping already-loac
Depth limit	A new kMaxIncludeDepth = 64 guard prevents stack-exhaustion crashes o
Error attribution	Module not found errors now include the importing file: <font face="Courier
--quiet / -q flag	bantu --quiet run server.b suppresses informational logs. Errors still print to
$BANTU_PATH search path	When a module can't be found via the standard resolution order, include no

1.4 Foundation Features (inherited from v1.2.1)

Feature	Description
include keyword	Language-level module imports. include "./routes.b"; brings symbols into sc
Real PostgreSQL driver	When compiled with -DBANTU_POSTGRES=ON, sua.postgres.*
Real MySQL driver	Same pattern via -DBANTU_MYSQL=ON + libmysqlclient.
Real WebRTC engine	-DBANTU_WEBRTC=ON + libdatachannel exposes sua.webrtc.*
VSCode extension	Syntax highlighting (TextMate grammar), context-aware autocomplete, hover hints, Go-to-Symbol, snipp
bantu build-windows	One command generates an NSIS .exe installer for the current Bantu project.
bantu bench	Built-in micro-benchmark suite for tracking interpreter regressions across releases.
Three sample apps	blogsite (modular Sua + SQLite), webrtc-chat (signaling + browser UI), pg-dashboa

2. Getting Started — Install & First Project

Bantu runs on any 64-bit Linux or Windows machine. The interpreter is a single static binary with no runtime dependencies — no Node, no Python, no DLLs. Installation is a two-step process: copy the binary somewhere, then run **bantu setup** to put it on PATH. After that, **bantu init myproject** scaffolds a working project in under a second.

2.1 Install on Linux / macOS

install-linux.sh

```
# 1. Download or build bantu
curl -L -o bantu https://github.com/AsseySilivestir/Bantu/releases/download/v1.2.2/bantu-linux
chmod +x bantu

# 2. Add to PATH (one-time)
./bantu setup            # user install (~/.local/bin)
sudo ./bantu setup --system   # system-wide (/usr/local/bin)

# 3. Verify
bantu --version
# -> Bantu v1.2.2

# 4. Seed the local package registry with starter packages
bantu setup --seed
```

2.2 Install on Windows

On Windows, download **bantu.exe** from the GitHub release page, drop it in any folder (e.g. **C:\Tools\bantu.exe**), then run **bantu.exe setup** from a Command Prompt. Bantu will add itself to your user PATH and you can immediately run **bantu init** from anywhere.

install-windows.bat

```
C:\Tools> bantu.exe setup
C:\Tools> bantu --version
Bantu v1.2.2

C:\Users\you> bantu init myproject
   Creating Bantu project: myproject
   - Created myproject/main.b
   - Created myproject/bantu.json
   Project ready!  cd myproject && bantu run
```

2.3 Diagnose Install Issues

If **bantu** is not found after install, run **bantu doctor** from the folder where the binary lives. It will check PATH, the local package registry, and the bantu.json manifest in the current directory, then print a diagnostic report.

bantu-doctor.txt

```
$ bantu doctor
  Bantu v1.2.2
  --------------------------------
  Binary:       /home/you/.local/bin/bantu  OK
  PATH:         ~/.local/bin is in PATH     OK
  Registry:     ~/.bantu/registry (5 pkgs)  OK
  Project:      ./bantu.json found          OK
  SQLite:       embedded                    OK
  PostgreSQL:   stub (compile -DBANTU_POSTGRES=ON for libpq)
  MySQL:        stub (compile -DBANTU_MYSQL=ON for mysqlclient)
  WebRTC:       stub (compile -DBANTU_WEBRTC=ON for libdatachannel)
```

2.4 Create a Project

Bantu ships two scaffolders. **bantu init <name>** creates a CLI project (a single **main.b** that prints "Hello, Bantu!"). **bantu init --web <name>** creates a full Sua web app with a routes file, controller file, database file, public/ folder, and a bantu.json manifest — the Spring Initializer of Bantu.

bantu-init.sh

```
$ bantu init myproject
   Creating Bantu project: myproject
   - myproject/main.b
   - myproject/bantu.json

$ cd myproject && bantu run
  Running: main.b
  ----------------------
  Hello, Bantu!

$ bantu init --web shop
   Creating Sua web app: shop
   - shop/server.b
   - shop/routes.b
   - shop/controller.b
   - shop/db.b
   - shop/public/index.html
   - shop/bantu.json
   Web app ready!  cd shop && bantu run server.b
```

2.5 Build from Source (for v1.2.2 with real drivers)

The prebuilt binary ships with stub drivers (deterministic, offline-friendly). To get real PostgreSQL, MySQL, and WebRTC, build from source with the appropriate CMake flags. The build is fast — about 20 seconds on a modern laptop.

build-from-source.sh

```
git clone https://github.com/AsseySilivestir/Bantu.git
cd Bantu/bantu-src/compiler
mkdir build && cd build

# Default build (stub drivers, ~660 KB binary)
cmake .. -DCMAKE_BUILD_TYPE=Release
make -j$(nproc)

# Full build with all real drivers
cmake .. -DCMAKE_BUILD_TYPE=Release \
         -DBANTU_POSTGRES=ON \
         -DBANTU_MYSQL=ON \
         -DBANTU_WEBRTC=ON
make -j$(nproc)

sudo cp bantu /usr/local/bin/
bantu --version   # -> Bantu v1.2.2
```

3. Language Syntax & Semantics

Bantu is a C-family language. Statements end with semicolons, blocks are delimited by curly braces, and variables are prefixed with **$** (inspired by PHP). The type system is dynamic with optional type annotations for documentation. This chapter walks through every syntactic construct in the language.

3.1 Variables

All variables are prefixed with **$**. The first assignment declares the variable; subsequent assignments mutate it. An optional type annotation (**number**, **string**, **bool**, **list**, **dict**, **any**, **func**) may precede the declaration as documentation — the runtime does not enforce it.

variables.b

```
// Untyped declarations
$name = "Alice";
$age = 30;
$active = true;
$scores = [90, 85, 78];
$config = { "host": "localhost", "port": 3000 };

// Typed declarations (documentation only)
number $count = 0;
string $greeting = "Hello";
bool   $enabled = false;
list   $items = [];
dict   $meta = {};
any    $dyn = null;
func   $handler = null;

// Constants
const $PI = 3.14159;
const $VERSION = "1.2.2";
```

3.2 Operators

Bantu supports the standard arithmetic, comparison, and logical operators. The dot operator **.** is used for both property access and method calls on objects.

operators.b

```
$a = 10 + 3;     // 13
$b = 10 - 3;     // 7
$c = 10 * 3;     // 30
$d = 10 / 3;     // 3.333...
$e = 10 % 3;     // 1

// Comparison
$f = ($a == 13);  // true
$g = ($a != 13);  // false
$h = ($a > 5);    // true

// Logical
$i = true and false;  // false
$j = true or false;   // true
$k = !true;            // false

// String concat
$l = "Hello" + ", " + "World";  // "Hello, World"

// Property/method access
$m = $config.host;            // "localhost"
$n = $items.push(99);         // appends 99
```

3.3 Functions

Functions are defined with the **def** keyword and use **$**-prefixed parameters. Functions are first-class values — they can be stored in variables, passed as arguments, and returned from other functions. The **return** statement sends a value back to the caller; without it, the function returns **null**.

functions.b

```
def greet($name) {
    print("Hello, " + $name + "!");
    return true;
}

// Default parameters
def greetFormal($name, $title = "Mr.") {
    return "Hello, " + $title + " " + $name;
}

// Higher-order functions
def applyTwice($f, $x) {
    return $f($f($x));
}

$double = def($x) { return $x * 2; };
print(applyTwice($double, 5));   // 20

// Anonymous (lambda) functions
$nums = [1, 2, 3, 4, 5];
$squared = $nums.map(def($x) { return $x * $x; });
print($squared);  // [1, 4, 9, 16, 25]
```

3.4 Control Flow

Bantu supports the usual control-flow constructs: **if/elif/else**, **while**, **for**, **each..in**, **switch/case**, **try/catch**, **break**, and **continue**. Loops can be nested arbitrarily and may be labeled for **break N** multi-level breaks.

control-flow.b

```
// if / elif / else
if ($score >= 90) {
    print("A");
} elif ($score >= 80) {
    print("B");
} else {
    print("C or below");
}

// while
$i = 0;
while ($i < 5) {
    print($i);
    $i += 1;
}

// for (C-style)
for ($j = 0; $j < 3; $j += 1) {
    print("j = " + $j);
}

// each..in (list iteration)
$colors = ["red", "green", "blue"];
each ($c in $colors) {
    print($c);
}

// switch
switch ($day) {
    case "Mon": print("Start of week"); break;
    case "Fri": print("Almost weekend"); break;
    default:    print("Midweek");
}

// try / catch
try {
    $data = sua.fetch("https://api.example.com/data");
} catch ($err) {
    print("Fetch failed: " + $err);
}
```

3.5 Classes & Objects

Bantu supports single-inheritance object-oriented programming via the **class**, **extends**, **init**, and **this** keywords. Methods are defined with **func** inside the class body. Visibility modifiers **private** and **public** are recognized as documentation — the runtime does not enforce them.

classes.b

```
class Animal {
    init($name, $sound) {
        this.name = $name;
        this.sound = $sound;
    }

    func speak() {
        print(this.name + " says " + this.sound);
    }

    func toString() {
        return "Animal(" + this.name + ")";
    }
}

class Dog extends Animal {
    init($name) {
        super.init($name, "Woof");
    }

    func fetch() {
        print(this.name + " fetches the stick.");
    }
}

$rex = new Dog("Rex");
$rex.speak();    // Rex says Woof
$rex.fetch();    // Rex fetches the stick.
print($rex.toString());  // Animal(Rex)
```

3.6 Data Structures

Bantu has two compound data types: **lists** (ordered, indexed) and **dicts** (key/value maps). Both support literal syntax, indexing, and a set of built-in methods. Lists use square brackets; dicts use curly braces with string keys.

data-structures.b

```
// Lists
$nums = [1, 2, 3, 4, 5];
$first = $nums[0];          // 1
$nums.push(6);              // [1,2,3,4,5,6]
$len = $nums.size();        // 6
$slice = $nums.slice(1, 3); // [2, 3]
$has3 = $nums.contains(3);  // true

// Dicts
$person = {
    "name": "Alice",
    "age": 30,
    "hobbies": ["reading", "hiking"]
};
$name = $person["name"];    // "Alice"
$person["email"] = "a@x";   // add a key
$keys = $person.keys();     // ["name", "age", "hobbies", "email"]
$has = $person.has("age");  // true

// Nested access
$hobby = $person["hobbies"][0];  // "reading"
```

3.7 Error Handling

Use **try** / **catch** to catch runtime errors. The **throw** keyword raises an error with a custom message. Errors that propagate uncaught abort the program with a stack trace. The **finally** block runs whether or not an error was raised.

error-handling.b

```
def divide($a, $b) {
    if ($b == 0) {
        throw "Division by zero";
    }
    return $a / $b;
}

try {
    $result = divide(10, 0);
} catch ($err) {
    print("Caught: " + $err);
    // -> Caught: Division by zero
} finally {
    print("Cleanup runs regardless.");
}
```

4. Module System — the include keyword [v1.2.2 NEW]

The **include** keyword is Bantu's language-level module system. It lets you split a project into multiple files and pull in symbols from them. There are two forms: **direct include** (brings all top-level symbols into the current scope) and **namespaced include** (binds them under a prefix). Path resolution is relative to the file doing the including, with cycle detection and a depth limit to prevent infinite recursion.

4.1 Direct Include

The simplest form. Symbols from the included file are available as if they were defined in the current file. Use this when the included file is small and its symbols won't collide with yours.

direct-include.b

```
// db.b - defines database helpers
def initDb() {
    sua.sqlite.connect("app.db");
    sua.sqlite.exec("CREATE TABLE IF NOT EXISTS users (...)");
}

def listUsers() {
    return sua.sqlite.query("SELECT * FROM users");
}

// main.b - includes db.b directly
include "./db.b";

initDb();              // initDb is now in scope
$users = listUsers();  // so is listUsers
print($users);
```

4.2 Namespaced Include

When you want to keep symbols organized, use the **as** form. All top-level symbols from the included file are accessed through the namespace prefix, preventing collisions.

namespaced-include.b

```
// routes.b
def registerAll($sua) {
    $sua.server.get("/api/health", def($req, $res) {
        $res.json({ "ok": true });
    });
    $sua.server.get("/api/users", def($req, $res) {
        $res.json(listUsers());
    });
}

// server.b - includes routes.b under a namespace
include "./routes.b" as routes;
include "./db.b";

initDb();
routes.registerAll(sua);    // access via routes. prefix
sua.server.listen(3000);
```

4.3 Path Resolution & $BANTU_PATH

Include paths are resolved in this order:

1. Relative to the file doing the include (e.g. `./db.b` from `server.b` looks for `server.b`'s sibling).
2. Canonicalized via `realpath()` (POSIX) or `GetFullPathName` (Windows) — so `./db.b` and `../app/db.b` resolve to the same canonical key, preventing double-execution.
3. Falls back to each directory in **$BANTU_PATH** (POSIX **:**-separated, Windows **;**-separated).

bantu-path.sh

```
# Enable shared module libraries outside the project tree
export BANTU_PATH=/opt/bantu/lib:~/bantu-modules:./vendor

# Now `include "logger.b";` will find /opt/bantu/lib/logger.b if it isn't
# already in the current project.
```

4.4 Cycle Detection & Depth Limit

Bantu guards against circular includes. If **a.b** includes **b.b** which includes **a.b** again, the second include is silently skipped (the module is already loaded) with a diagnostic:

cycle-detection.txt

```
$ bantu run server.b
  [INCLUDE] Loaded ./db.b
  [INCLUDE] Loaded ./routes.b
  Skipping already-loaded module: ./db.b
  Server listening on port 3000
```

A hard depth limit of **kMaxIncludeDepth = 64** prevents pathological self-generating chains from exhausting the stack. If you hit it, Bantu prints `[INCLUDE ERROR] Maximum include depth (64) exceeded` and aborts.

4.5 --quiet mode

For production logs, the **--quiet** / **-q** flag suppresses the `[INCLUDE]` and `[Executed in ... us]` informational lines. Errors still go to stderr.

quiet-mode.txt

```
$ bantu --quiet run server.b
  Server listening on port 3000
  (no other output unless an error occurs)
```

5. Sua Web Framework — HTTP Server & Client

The **sua** namespace is Bantu's built-in web framework. It exposes an HTTP server (**sua.server**), an HTTP client (**sua.fetch**), JSON helpers (**sua.json**), and the database drivers (**sua.sqlite**, **sua.postgres**, **sua.mysql**). The HTTP server runs on native C++ sockets — there is no Node-style event loop overhead.

5.1 HTTP Server

The server is route-based: register handlers with **sua.server.get**, **sua.server.post**, etc., then call **sua.server.listen(port)** to start. Handlers receive a **$req** (request) and **$res** (response) object.

server.b

```
// server.b
sua.server.get("/", def($req, $res) {
    $res.html("<h1>Hello from Bantu!</h1>");
});

sua.server.get("/api/health", def($req, $res) {
    $res.json({ "ok": true, "version": "1.2.2" });
});

sua.server.post("/api/users", def($req, $res) {
    $name = $req.body["name"];
    $email = $req.body["email"];
    // ... insert into db ...
    $res.status(201);
    $res.json({ "id": 42, "name": $name });
});

sua.server.static("/static", "./public");
sua.server.listen(3000);
print("Server on http://localhost:3000");
```

5.2 Request & Response Objects

The **$req** object exposes the HTTP method, path, headers, query string, and parsed body. The **$res** object lets you set status, headers, and send the body in various formats.

req-res.b

```
sua.server.post("/api/echo", def($req, $res) {
    // $req fields
    $method = $req.method;       // "POST"
    $path   = $req.path;         // "/api/echo"
    $headers = $req.headers;     // dict
    $query  = $req.query;        // dict (parsed query string)
    $body   = $req.body;         // dict (parsed JSON body)

    // $res methods
    $res.status(200);
    $res.header("X-Custom", "yes");
    $res.json({
        "received": $body,
        "method": $method,
        "path": $path
    });
});
```

5.3 HTTP Client (sua.fetch)

The **sua.fetch** function makes HTTP requests. It supports GET, POST, PUT, DELETE, custom headers, and JSON bodies. Returns a response dict with **status**, **headers**, and **body** keys.

fetch.b

```
// GET request
$resp = sua.fetch("https://api.github.com/repos/AsseySilivestir/Bantu");
print($resp.status);                  // 200
$data = sua.json.parse($resp.body);   // dict
print($data["full_name"]);            // "AsseySilivestir/Bantu"
print($data["stargazers_count"]);

// POST request with JSON body
$resp = sua.fetch("https://httpbin.org/post", {
    "method": "POST",
    "headers": { "Content-Type": "application/json" },
    "body": sua.json.stringify({ "name": "Alice", "age": 30 })
});
print($resp.status);   // 200
```

5.4 Static Files

Serve static files from a directory with **sua.server.static**. The first argument is the URL prefix; the second is the filesystem path. MIME types are auto-detected from file extensions.

static-files.b

```
// Serve anything in ./public/ under /static/
sua.server.static("/static", "./public");

// Now /static/index.html serves ./public/index.html
// /static/css/style.css serves ./public/css/style.css
```

5.5 JSON Helpers

json.b

```
// Parse a JSON string into a Bantu dict/list
$obj = sua.json.parse('{"name":"Alice","scores":[90,85]}');
print($obj["name"]);             // "Alice"
print($obj["scores"][0]);        // 90

// Stringify a Bantu value back to JSON
$json = sua.json.stringify({ "ok": true, "items": [1, 2, 3] });
print($json);  // '{"ok":true,"items":[1,2,3]}'

// Pretty-print with 2-space indent
$pretty = sua.json.stringify($obj, 2);
```

6. Database Drivers — SQLite, PostgreSQL, MySQL

Bantu ships three database drivers under the **sua** namespace. SQLite is always available — it's statically linked into the binary. PostgreSQL and MySQL are compile-time options: the default build ships deterministic stubs (so your code doesn't crash on machines without libpq/libmysqlclient); build with **-DBANTU_POSTGRES=ON** or **-DBANTU_MYSQL=ON** to enable real drivers.

6.1 SQLite (always available)

sqlite.b

```
// Connect (creates the file if it doesn't exist)
sua.sqlite.connect("app.db");

// Schema
sua.sqlite.exec("CREATE TABLE IF NOT EXISTS users ("
              + "  id INTEGER PRIMARY KEY AUTOINCREMENT,"
              + "  name TEXT NOT NULL,"
              + "  email TEXT UNIQUE,"
              + "  created_at TEXT DEFAULT CURRENT_TIMESTAMP"
              + ")");

// Insert
sua.sqlite.exec("INSERT INTO users (name, email) VALUES ('Alice', 'a@x.com')");

// Parameterized query (safe from SQL injection)
sua.sqlite.exec("INSERT INTO users (name, email) VALUES (?, ?)",
                ["Bob", "b@x.com"]);

// Query -> list of dicts
$rows = sua.sqlite.query("SELECT id, name, email FROM users");
each ($row in $rows) {
    print($row["id"] + ": " + $row["name"] + " <" + $row["email"] + ">");
}

sua.sqlite.close();
```

6.2 PostgreSQL (real with -DBANTU_POSTGRES=ON)

postgres.b

```
// Connect with a libpq connection string
sua.postgres.connect(
    "host=localhost dbname=app user=postgres password=secret"
);

// Schema
sua.postgres.exec("CREATE TABLE IF NOT EXISTS users ("
    + "  id SERIAL PRIMARY KEY,"
    + "  name TEXT NOT NULL,"
    + "  email TEXT UNIQUE"
    + ")");

// Parameterized (PG-style $1, $2)
sua.postgres.exec(
    "INSERT INTO users (name, email) VALUES ($1, $2)",
    ["Carol", "c@x.com"]
);

// Query
$rows = sua.postgres.query("SELECT * FROM users");
print($rows.size() + " users");

sua.postgres.close();
```

6.3 MySQL (real with -DBANTU_MYSQL=ON)

mysql.b

```
// Connect: host, user, password, database, port
sua.mysql.connect("localhost", "root", "secret", "app", 3306);

sua.mysql.exec("CREATE TABLE IF NOT EXISTS users ("
    + "  id INT AUTO_INCREMENT PRIMARY KEY,"
    + "  name VARCHAR(100) NOT NULL,"
    + "  email VARCHAR(255) UNIQUE"
    + ")");

// Parameterized (?-style)
sua.mysql.exec(
    "INSERT INTO users (name, email) VALUES (?, ?)",
    ["Dave", "d@x.com"]
);

$rows = sua.mysql.query("SELECT * FROM users");
print($rows);

sua.mysql.close();
```

6.4 Detecting Real vs Stub Drivers

At runtime, you can ask a driver whether it's real or stub via the **.real** boolean property. This is useful when your app needs to gracefully degrade on machines without the optional libraries.

driver-check.b

```
if (!sua.postgres.real) {
    print("[WARN] PostgreSQL driver is a stub. Falling back to SQLite.");
    $db = "sqlite";
} else {
    sua.postgres.connect("host=localhost dbname=app");
    $db = "postgres";
}
```

7. WebRTC Engine — Peer Connections & Data Channels [v1.2.2 NEW]

The **sua.webrtc** namespace exposes a WebRTC engine for peer-to-peer real-time communication. The default build ships a stub that records API calls but doesn't actually open peer connections. Build with **-DBANTU_WEBRTC=ON** (links libdatachannel) to get real peer-to-peer audio, video, and data channels.

7.1 Peer Connection Lifecycle

webrtc-peer.b

```
// Create a peer
$peer = sua.webrtc.peer("alice");

// Create an SDP offer
$offer = sua.webrtc.createOffer("alice");
print($offer);   // SDP string

// (Send $offer to the remote peer via your signaling layer —
//  e.g. WebSocket, HTTP POST, or Bantu's relay server)

// Receive the remote answer
sua.webrtc.setRemoteAnswer("alice", $remoteAnswer);

// Add ICE candidates as they arrive
sua.webrtc.addIceCandidate("alice", $candidate);
```

7.2 Data Channels

Data channels are bidirectional, ordered, message-oriented streams — perfect for chat, remote control, or sync protocols.

webrtc-channel.b

```
// Open a data channel on a connected peer
$chan = sua.webrtc.dataChannel("chat");

// Send a message
sua.webrtc.send("chat", "Hello from Bantu!");

// Register a handler for incoming messages
sua.webrtc.onMessage("chat", def($msg) {
    print("Received: " + $msg);
});

// Close
sua.webrtc.close("chat");
```

7.3 Signaling Server (bantu relay)

Bantu ships a built-in STUN/TURN relay server for NAT traversal. Start it with **bantu relay** on a machine with a public IP, and your Bantu WebRTC apps can use it for ICE candidate exchange.

relay.txt

```
# On a public server:
$ bantu relay 3478

  ------------------------------------------------
  Bantu WebRTC Relay Server v1.2.2
  ------------------------------------------------
  STUN server:    UDP port 3478
  TURN relay:     UDP port 3479
  Relay ports:    50000-60000
  Signaling:      SDP offer/answer
  ICE:            Candidate gathering
  ------------------------------------------------
```

7.4 Sample: Two-Browser Chat

The repository ships a complete WebRTC chat sample at **samples/webrtc-chat/**. It demonstrates a Bantu signaling server that exchanges SDP offers/answers between two browser tabs. See **samples/webrtc-chat/README.md** for the walkthrough.

8. VSCode Extension — Highlighting, Hints, Blue-B Icon [v1.2.2 NEW]

Bantu ships a first-class VSCode extension (**bantu-vscode-1.2.2.vsix**) that turns VSCode into a Bantu IDE. It's a single ~24 KB VSIX file — no dependencies, no Language Server download. Install it offline and you immediately get: syntax highlighting, context-aware autocomplete, hover hints, Go-to-Symbol, snippets, the blue-B file icon for ***.b*** files, and one-click Run (F5).

8.1 Install

install-vsix.sh

```
# From the release zip (offline)
code --install-extension bantu-vscode-1.2.2.vsix

# Or from the VSCode marketplace (once published)
# Search "Bantu" in the Extensions panel
```

8.2 Features

Feature	What it does
Syntax highlighting	TextMate grammar covering every Bantu keyword, type, operator, and string form. Comments, strings,
Autocomplete	Context-aware completion for keywords, types, sua.* namespaces and methods, project-define
Hover hints	Hover over any sua.* method to see its signature and a short example. Hover over a keyword t
Go-to-Symbol	Cmd/Ctrl+Shift+O opens the symbol panel — lists every function, class, and top-level definition
Snippets	Type def + Tab to expand a function template. class + Tab for a class. sua.get
Blue-B file icon	Every *.b file shows a custom blue "B" icon in the explorer (light and dark variants).
One-click Run (F5)	Press F5 in any .b file to run it via bantu run. Output goes to the integrated terminal.

8.3 Configuration

The extension reads **bantu.json** in the project root for the entry file and Bantu version. You can also override the bantu binary path in VSCode settings:

vscode-settings.json

```
// .vscode/settings.json
{
    "bantu.binaryPath": "/usr/local/bin/bantu",
    "bantu.entryFile": "main.b",
    "bantu.runArgs": ["--quiet"]
}
```

8.4 Commands

The extension contributes the following commands to the Command Palette (**Cmd/Ctrl+Shift+P**):

Command	Description
Bantu: Run File	Run the current .b file with bantu run. Default key: F5.
Bantu: Initialize Project	Run bantu init in the workspace folder.
Bantu: Initialize Sua Web Project	Run bantu init --web in the workspace.
Bantu: Build Windows Installer	Run bantu build-windows (Windows only).
Bantu: Build Desktop Installer	Run bantu installer (cross-platform).

9. Building Desktop Installers — bantu installer [v1.2.2 NEW]

The **bantu installer** command packages any Bantu project into a single, cross-platform desktop installer. The installer bundles the bantu interpreter itself — so end users can run your app on machines that don't have Bantu installed. Ship the installer to a friend, they double-click it, the app appears in their launcher menu. No internet required, no install step beyond the installer itself.

Why this matters. Before v1.2.2, distributing a Bantu app meant asking the user to install Bantu first, then handing them a **.b** file. With **bantu installer**, the app is a self-contained **.deb** / **.exe** / **.app** — exactly what users expect from a desktop app.

9.1 Quick Start

From inside a Bantu project (one with a **main.b**, **app.b**, **index.b**, or **server.b** entry file):

installer-quickstart.txt

```
$ bantu installer

  ------------------------------------------------
  bantu installer - desktop installer builder
  ------------------------------------------------
  App:      myapp v1.0.0
  Entry:    main.b
  Platform: linux           (auto-detected)
  Bundle:   yes (embed bantu)

  [INSTALLER] Bundled interpreter: /home/you/.local/bin/bantu
  [INSTALLER] Building .deb: myapp_1.0.0_amd64.deb

  [INSTALLER] Linux installer ready:
    dist/myapp_1.0.0_amd64.deb

  Install on any Debian/Ubuntu machine:
    sudo dpkg -i dist/myapp_1.0.0_amd64.deb
    myapp                  # run from terminal
    # Or find 'myapp' in your application launcher.
```

9.2 Cross-Platform Targets

Bantu installer auto-detects the host platform by default. To cross-build for another platform, pass **--platform**. The three supported targets:

Platform	Command	Output	Distribution
Linux	bantu installer app.b --platform linux	dist/<name>_<ver>_amd64.deb	Debian / Ubuntu / Mint. Installs via dpkg -i. Adds a <
Windows	bantu installer app.b --platform windows	dist/<name>-Setup-<ver>.exe	Installer. Installs to %LOCALAPPDATA%\<name
macOS	bantu installer app.b --platform macos	dist/<Name>.app	Standard macOS .app bundle. Double-click to run, or use <b

9.3 Options

Option	Default	Description
[entry.b]	(auto)	Entry file. Auto-detected in this order: main.b, app.b, index.b, server.b.
--name <Name>	from bantu.json or folder name	Application display name. Used in package name, Start Menu folder, .app bundle nam
--version <x.y.z>	from bantu.json or "1.0.0"	Application version. Used in package filename and .deb control / Info.plist.
--platform <p>	auto-detect	One of linux, windows, macos. Shorthands win,
--icon <path>	(none)	Path to an icon file. Linux: .png (256x256 ideal). Windows: .ico. macO
--bundle-bantu	on	Embed the bantu interpreter inside the installer. Default: on. Use --no-bun

9.4 Project Configuration (bantu.json)

The installer reads **name** and **version** from **bantu.json** if you don't pass them on the command line. A typical manifest:

bantu.json

```
{
    "name": "myapp",
    "version": "1.2.0",
    "entry": "main.b",
    "language": "bantu",
    "bantuVersion": "1.2.2",
    "description": "A desktop todo app written in Bantu"
}
```

9.5 Linux .deb Internals

The Linux installer is a standard Debian package. The layout Bantu produces:

deb-layout.txt

```
myapp_1.0.0_amd64.deb
■■■ usr/
■   ■■■ bin/
■   ■   ■■■ myapp                    # launcher script (on PATH)
■   ■■■ lib/myapp/
■   ■   ■■■ bantu                    # bundled interpreter
■   ■   ■■■ main.b                   # your entry file
■   ■   ■■■ *.b                      # all other Bantu source files
■   ■   ■■■ bantu.json               # project manifest
■   ■   ■■■ public/                  # static assets (if present)
■   ■■■ share/
■       ■■■ applications/
■       ■   ■■■ myapp.desktop        # launcher menu entry
■       ■■■ icons/hicolor/256x256/apps/
■           ■■■ myapp.png            # (if --icon was passed)
■■■ DEBIAN/
    ■■■ control                     # package metadata
    ■■■ postinst                    # fix permissions after unpack
    ■■■ prerm                       # pre-remove hook
```

The launcher script at **/usr/bin/myapp** is a tiny bash wrapper that finds the bundled bantu (or falls back to a system bantu if present) and runs your entry file:

usr-bin-myapp

```
#!/bin/bash
# Generated by bantu installer v1.2.2
# App: myapp v1.0.0
set -e
APP_LIB="/usr/lib/myapp"
if [ ! -d "$APP_LIB" ]; then
    APP_LIB="$HOME/.local/lib/myapp"
fi
if [ -x "$APP_LIB/bantu" ]; then
    BANTU_BIN="$APP_LIB/bantu"
elif command -v bantu >/dev/null 2>&1; then
    BANTU_BIN="bantu"
else
    echo "[ERROR] Bantu interpreter not found." >&2
    exit 1
fi
cd "$APP_LIB"
exec "$BANTU_BIN" run main.b "$@"
```

9.6 Windows .exe Internals

The Windows installer is an NSIS script that, when compiled with **makensis**, produces a self-contained **.exe** installer. The script handles install, uninstall, and Start Menu shortcut creation. Key behaviors:

Install dir: %LOCALAPPDATA%\<AppName> (no admin rights needed).
Bundled files: bantu.exe, all ***.b** source files, **bantu.json**, any ***.bat** helpers, and the icon (if passed).
Launcher: a **run-<App>.bat** is generated that **cd**s into the install dir and calls **bantu.exe run <entry>**.
Shortcuts: Start Menu entry under **%SMPROGRAMS%\<App>**, plus an uninstall shortcut.
Uninstaller: writes a **uninstall-<App>.exe** and registers it via **WriteUninstaller**.

If **makensis** is not on PATH, Bantu still writes the **.nsi** file — you can compile it later on any machine (Windows, Linux, or macOS) with NSIS installed:

makensis.sh

```
# Linux: install NSIS once
sudo apt install nsis

# Then compile the script Bantu generated
makensis dist/installer-myapp.nsi

# Output: dist/myapp-Setup-1.0.0.exe
```

9.7 macOS .app Internals

The macOS installer is a standard **.app** bundle you can double-click to run, drag into **/Applications/**, or package into a **.dmg** for distribution.

app-layout.txt

```
myapp.app/
■■■ Contents/
    ■■■ Info.plist              # bundle metadata (CFBundleName, version, etc.)
    ■■■ PkgInfo                 # 8-byte APPL???? signature
    ■■■ MacOS/
    ■   ■■■ myapp               # launcher script (the CFBundleExecutable)
    ■   ■■■ bantu              # bundled interpreter
    ■■■ Resources/
        ■■■ main.b              # your entry file
        ■■■ *.b                 # all other Bantu source
        ■■■ bantu.json
        ■■■ public/             # static assets (if present)
        ■■■ app.icns            # icon (if --icon was passed)
```

To distribute as a **.dmg** (the standard macOS download format), wrap the .app with **hdiutil** on a Mac:

make-dmg.sh

```
hdiutil create \
    -volname 'MyApp' \
    -srcfolder dist/myapp.app \
    -ov -format UDZO \
    dist/myapp-1.0.0.dmg
```

9.8 End-to-End Example: Distributing a Bantu Web App

Say you've written a Sua web app and want to share it with a friend on Ubuntu. They don't have Bantu installed and don't want to install Node or Python. Here's the full workflow:

e2e-example.sh

```
# 1. Build the installer
$ cd my-sua-app
$ bantu installer --platform linux --name "MyApp" --version "1.0.0"
  ...
  [INSTALLER] Linux installer ready:
    dist/myapp_1.0.0_amd64.deb

# 2. Send dist/myapp_1.0.0_amd64.deb to your friend (e.g. via email,
#    USB stick, file share - it's just 800 KB).

# 3. On the friend's Ubuntu machine:
$ sudo dpkg -i myapp_1.0.0_amd64.deb
$ myapp
  Server on http://localhost:3000

# Or: open the application launcher and click "MyApp".
```

9.9 Uninstall

Each platform has a clean uninstall path:

Platform	Uninstall command
Linux	sudo dpkg -r <pkgname> (or use your distro's package manager GUI).
Windows	Run uninstall-<App>.exe from the install dir, or use Add/Remove Programs in

Platform	Uninstall command
macOS	Drag the .app to the Trash.

9.10 Comparison: bantu installer vs bantu build-windows

Bantu v1.2.1 shipped **bantu build-windows**, which produces a Windows-only NSIS installer. The new **bantu installer** is its successor: cross-platform, bundles the bantu interpreter by default, reads defaults from **bantu.json**, and supports icons. The old command is kept for backwards compatibility but new projects should use **bantu installer**.

Feature	bantu build-windows	bantu installer
Platforms	Windows only	Linux, Windows, macOS
Bundles bantu	Optional	Yes (default; can disable with --no-bundle-bantu)
Reads bantu.json	No	Yes (name, version)
Custom icon	No	Yes (--icon)
Auto-detect platform	n/a	Yes
Generated output	Always .exe	.deb / .exe / .app

10. Building Windows Installers — bantu build-windows

The **bantu build-windows** command (introduced in v1.2.1, retained in v1.2.2 for backwards compatibility) generates an NSIS-based Windows installer from the current Bantu project. For new projects, prefer the cross-platform **bantu installer** command (Chapter 9) — it does everything **build-windows** does, plus Linux and macOS, and bundles the interpreter by default.

10.1 Usage

build-windows.txt

```
# Basic: defaults to bantu.json or folder name
$ bantu build-windows

# With options
$ bantu build-windows app.b --name "MyApp" --version "1.0.0"

  [BUILD-WINDOWS] NSIS script written: dist/installer.nsi
  [BUILD-WINDOWS] App: MyApp v1.0.0
  [BUILD-WINDOWS] Entry: app.b

  [BUILD-WINDOWS] Installer generated:
    dist/MyApp-Setup-1.0.0.exe
```

10.2 What it bundles

The NSIS script bundles all ***.b** files in the current directory, plus **bantu.json** and any ***.bat** helpers. If **bantu.exe** is in the current directory (or the build dir), it's bundled too — otherwise the installer assumes Bantu is already installed on the target machine.

10.3 Install experience on Windows

When the user runs **MyApp-Setup-1.0.0.exe**:

1. NSIS shows a standard install wizard — license, install dir, components.
2. Files are copied to `%LOCALAPPDATA%\MyApp` (no admin rights needed).
3. A Start Menu folder **MyApp** is created with a launch shortcut and an uninstall shortcut.
4. An uninstaller is registered in *Add/Remove Programs*.

11. Sample Applications

The Bantu repository ships three complete, runnable sample apps in **samples/**. Each demonstrates a different Bantu use case and is small enough to read end-to-end in a few minutes.

11.1 blogsite — Modular Sua + SQLite

A small blog with posts, comments, and an admin panel. Demonstrates the **include** keyword (splits routes, controller, db, and main server into separate files), SQLite persistence, and Sua's static file serving for the public HTML/CSS.

samples/blogsite/server.b

```
// samples/blogsite/server.b
include "./db.b";
include "./controller.b" as ctrl;
include "./routes.b" as routes;

initDb();
routes.registerAll(sua, ctrl);
sua.server.listen(3000);
```

11.2 webrtc-chat — Signaling + Browser UI

A two-browser WebRTC chat. The Bantu server acts as the signaling layer (exchanging SDP offers/answers and ICE candidates via HTTP), and the browser UI is a single HTML file with vanilla JavaScript. Demonstrates **sua.webrtc.*** and the relay server.

samples/webrtc-chat/server.b

```
// samples/webrtc-chat/server.b
sua.server.post("/offer", def($req, $res) {
    $from = $req.body["from"];
    $offer = $req.body["offer"];
    $offers[$from] = $offer;
    $res.json({ "ok": true });
});

sua.server.get("/poll-offer", def($req, $res) {
    $from = $req.query["from"];
    if ($offers.has($from)) {
        $res.json({ "offer": $offers[$from] });
        $offers.remove($from);
    } else {
        $res.json({ "offer": null });
    }
});

sua.server.listen(3000);
```

11.3 pg-dashboard — PostgreSQL Analytics

A read-only analytics dashboard backed by real PostgreSQL. Demonstrates the **sua.postgres.*** driver (requires building Bantu with **-DBANTU_POSTGRES=ON**) and serves a small HTML dashboard with charts.

samples/pg-dashboard/server.b

```
// samples/pg-dashboard/server.b
sua.postgres.connect("host=localhost dbname=analytics user=postgres");

sua.server.get("/", def($req, $res) {
    $stats = sua.postgres.query(
        "SELECT date_trunc('day', created_at) AS day, "
        + "COUNT(*) AS events FROM events "
        + "WHERE created_at > NOW() - INTERVAL '30 days' "
        + "GROUP BY day ORDER BY day"
    );
    $res.html(renderDashboard($stats));
});

sua.server.listen(3000);
```

11.4 Running the Samples

run-samples.sh

```
git clone https://github.com/AsseySilivestir/Bantu.git
cd Bantu

# blogsite (default build, SQLite)
cd samples/blogsite
bantu run server.b
# -> open http://localhost:3000

# webrtc-chat (default build, WebRTC stub is fine for signaling)
cd ../webrtc-chat
bantu run server.b
# -> open http://localhost:3000 in two browser tabs

# pg-dashboard (requires -DBANTU_POSTGRES=ON rebuild)
cd ../pg-dashboard
bantu run server.b
```

12. Command Reference

Every command Bantu v1.2.2 understands. Run **bantu --help** for the same list in your terminal.

12.1 Core Commands

Command	Description
bantu	Start the interactive REPL.
bantu run <file.b>	Run a Bantu file. bantu run (no arg) runs main.b.
bantu build <file.b>	Compile to a standalone executable (shell script that embeds the source).
bantu init <name>	Scaffold a CLI project (Hello World main.b).
bantu init --web <name>	Scaffold a Sua web app (server.b + routes.b + controller.b + db.b + public/).
bantu --version	Print Bantu version (currently 1.2.2).
bantu --help	Print this help.

12.2 PATH & Diagnostics

Command	Description
bantu setup	Add bantu to PATH (user install: ~/.local/bin).
bantu setup --system	Add bantu to PATH (system-wide: /usr/local/bin; needs sudo/admin).
bantu setup --seed	Also seed the local registry with starter packages.
bantu doctor	Diagnose install state, PATH, registry, manifest, and driver flags.
bantu uninstall	Remove bantu from PATH.

12.3 Package Manager

Command	Description
bantu install	Install all deps listed in bantu.json.
bantu add <pkg>	Add a dep to bantu.json and install it.
bantu add <pkg>@<ver>	Pin a specific version.
bantu remove <pkg>	Remove a dep and uninstall it.
bantu update	Update all deps to their latest version.
bantu update <pkg>	Update one dep.
bantu list	List deps installed in the current project.
bantu search [term]	Search the local registry (offline).
bantu publish <dir>	Add a folder to the local registry.
bantu publish <dir> --as <n>	Publish under a different name.

12.4 Desktop Installer Commands

Command	Description
bantu installer	Auto-detect host platform and build a desktop installer. Reads bantu.json for name/ve
bantu installer app.b --platform linux	Build a .deb package + .desktop launcher entry.
bantu installer app.b --platform windows	Generate an NSIS .exe installer (run makensis to compile).
bantu installer app.b --platform macos	Build a macOS .app bundle (use hdiutil to wrap into a .dmg).
bantu build-windows [opts] [entry.b]	Legacy Windows-only NSIS installer (v1.2.1). Prefer bantu installer.

12.5 Global Flags

Flag	Description
bantu --help / -h	Print the help screen.
bantu --version / -v	Print version.
bantu --quiet / -q	Suppress informational logs ([INCLUDE] Loaded ..., [Executed in ...]). Errors still go to stderr.

13. Roadmap & Community

13.1 v1.3 Roadmap

The v1.3 release is planned for late 2026 and focuses on performance and developer ergonomics. The headline feature is a register-based bytecode VM that should speed up tight CPU-bound loops significantly. Other planned items:

Planned	Description
Bytecode VM	Register-based bytecode compiler + VM. Targets a large speedup on tight loops.
LSP server	Full Language Server Protocol implementation for VSCode with diagnostics, jump-to-definition, rename refa
sua.crypto	Built-in crypto module: SHA-256, HMAC, AES-256-GCM, bcrypt password hashing.
sua.websocket	Native WebSocket server (currently only available via the WebRTC relay).
Async/await	First-class async / await on top of a libuv-style event loop.
Package registry	Public package registry at registry.bantu.dev with bantu publish --public.
ARM64 builds	Prebuilt binaries for Apple Silicon (macOS-arm64) and Linux ARM64 (Raspberry Pi 4/5).
REPL improvements	Multi-line input, command history, tab completion in the REPL.
RPM & Flatpak installers	Extend bantu installer to also produce .rpm and .flatpak packages for Fedora / Arc

13.2 Repository

Source code, samples, and this PDF live on GitHub:

clone.sh

```
https://github.com/AsseySilivestir/Bantu.git

Branches:
  main        - stable release tag (currently v1.2.2)
  develop     - v1.3 in progress

Clone and build:
  git clone https://github.com/AsseySilivestir/Bantu.git
  cd Bantu/bantu-src/compiler
  mkdir build && cd build
  cmake .. -DCMAKE_BUILD_TYPE=Release
  make -j$(nproc)
```

13.3 Contributing

Contributions are welcome. Open an issue first to discuss the change before opening a PR. Areas that particularly need help:

Area	How to help
Real drivers	Test -DBANTU_POSTGRES=ON / -DBANTU_MYSQL=ON / -DBANTU_WEBRTC=ON
Sample apps	Write a 4th sample: a CLI tool, a Discord bot, or a static site generator.

Area	How to help
Installer coverage	Test bantu installer on more distros (Fedora, Arch, NixOS) and macOS versions. Help add RPM / Flat
VSCode extension	Help build the LSP server (v1.3 roadmap). Test the existing extension on Windows.
Documentation	Fix typos, expand examples, translate sections to other languages.
Bug reports	Open an issue with a minimal reproducer — see CONTRIBUTING.md for the template.

13.4 License & Attribution

Bantu is released under the MIT License. See **LICENSE** in the repository root. The interpreter uses libsqlite3 (public domain), libcurl (MIT-style), and optionally libpq (PostgreSQL License), libmysqlclient (GPLv2 with FOSS exception), and libdatachannel (LGPLv2) — all of which are compatible with MIT distribution.

Bantu was created by **Assey Silivestir Peter**. The language is named after the Bantu language family spoken across sub-Saharan Africa — a nod to the developer's Tanzanian roots and the language's goal of being accessible to developers in low-bandwidth, offline-first environments.

13.5 Citation

citation.bib

```
@misc{bantu-v1.2.2,
  author    = {Assey Silivestir Peter},
  title     = {Bantu Programming Language v1.2.2},
  year      = {2026},
  publisher = {GitHub},
  url       = {https://github.com/AsseySilivestir/Bantu}
}
```